

Lab Report: Red-Black Trees and XC3-Trees

Course: 2XC3 - Algorithms and Data Structures

Prepared By:

Ben Guo
Heeman Ijaz
Jason Xie

Submitted To:

Dr. Ilir Dema
Department of Computing and Software
McMaster University

March 12, 2026

Contents

Executive Summary	1
1 RBT vs BST Height on Random Inputs	2
1.1 Objective	2
1.2 Methodology	2
1.3 Results	2
1.3.1 When is a BST preferred over an RBT?	2
2 RBT vs BST Height Relative to Number of Swaps	4
2.1 Objective	4
2.2 Methodology	4
2.3 Results	4
3 XC3-Tree Height $h(i)$ for Degrees 0 to 25	6
3.1 Objective	6
3.2 Methodology	6
3.3 Results	6
4 XC3-Tree Nodes $\text{nodes}(i)$ and Height $O(\log n)$	8
4.1 Objective	8
4.2 Methodology	8
4.3 Results	8
4.4 Argument: Why Height is $O(\log n)$	8
A Code Navigation Guide	10

List of Figures

1	Effect of Disorder on Tree Height Difference for 1000 nodes	5
2	XC3-Tree degree i vs height $h(i)$	7
3	XC3-Tree degree i vs nodes(i) (log scale)	9

Executive Summary

In this lab, we implement, test, and analyze self-balancing tree structures and their height behavior. First, we completed a Red-Black Tree (RBT) insertion implementation by adding the missing rotation operations and the rebalancing procedure after insertion. Then, we compared the heights of RBTs and standard Binary Search Trees (BSTs) under random inputs and under inputs that are close to sorted. Next, we implemented XC3-Trees and tested their height and node-growth patterns for degrees from 0 to 25. Finally, we used these results to support an argument that the height of an XC3-Tree is $O(\log n)$ with respect to the number of nodes n .

Key Takeaways

- **Understanding the difference between BSTs & RBTs:** Even though both store keys in a search-tree structure, RBTs enforce balancing rules that prevent extreme heights that BSTs can suffer from under certain inputs
- **Finding and analyzing properties of a data structure:** We discover relationships between input ordering (random vs nearly sorted) and the resulting height of BSTs compared to RBTs
- **Using rotations and recoloring to maintain invariants:** RBT insertion requires structural changes (rotations) and color changes (recoloring) to preserve the Red-Black rules after each insertion
- **XC3-Tree height and growth patterns:** By testing degrees 0 to 25, we identify consistent patterns for the height ($h(i)$) and number of nodes ($\text{nodes}(i)$), which leads to a logarithmic-height argument

1 RBT vs BST Height on Random Inputs

1.1 Objective

Our goal is to compare the heights of RBTs and BSTs when built from random lists of length 10,000, and analyze the average difference in height. Using this information, we will draw conclusions on what the specific uses cases are for both, and why we might want to use one over the other in certain situations.

1.2 Methodology

We generated random lists of length 10,000 and inserted them into both a BST and an RBT. For each trial we measured the height of each tree and computed the height difference. We repeated this across 100 trials and averaged the results to smooth out randomness. We then output the average height of RBT trees, BST trees, and the average difference between these two heights all calculated through these trials.

1.3 Results

- **Number of trials:** 100
- **Random value range/distribution:** *Input size is 10,000, each element is in the range of 1-100,000*
- **Metric:** $\Delta h = h(\text{BST}) - h(\text{RBT})$
- **Average RBT Height:** 16.14
- **Average BST Height:** 31.27
- **Average Difference:** 15.13

These results show that the RBT consistently maintains a much smaller height than the BST when inserting the same set of random values. Over all the trials, the average height of the RBT was 16.14, while the BST had an average height of 31.27, resulting in an average difference of 15.13 levels. Of course, our theoretical is bound is that the height of an RBT will be bounded at $2\log_2(n)$ no matter what. For $n = 10,000$, the limit is ≈ 27 , which we are well below. On the other hand, the worst case for a BST is $O(n)$, though the odds of this worst case are so low we will never experience this when we average out the results.

1.3.1 When is a BST preferred over an RBT?

The reason we may prefer a BST over an RBT is intuitively similar as to why we would prefer insertion sort over merge sort/quick sort. It comes down to whether the overhead (additional memory, complex implementation, potential debugging) of implementing and maintaining an RBT outweighs the benefits? For example, if in our experiment with lists of length 10,000 having twice the height slows down our required operations less than the amount of time it takes to maintain the RBT, it may be preferable to stick with a BST.

If the input is randomly ordered, a BST will naturally produce a reasonably balanced tree with expected height $O(\log n)$ (by the law of random numbers), meaning the self-balancing properties of an RBT provide little relative benefit. In this case, the simpler BST insert (a plain traversal) will be faster per operation. In addition, if the task is simpler such as making many lookups but not many insertions/deletions, a BST will perform very similarly to an RBT while also avoiding the associated overhead.

2 RBT vs BST Height Relative to Number of Swaps

2.1 Objective

Our goal is similar to the previous experiment however this time we are analyzing the connection between input “sortedness” and the height difference between BSTs and RBTs. We want to understand how the height changes as we go from a fully sorted input towards a more randomized input like we saw previously.

2.2 Methodology

Since our goal is similar, our methodology will also be similar. This time, we start from a completely sorted list and gradually make it more unsorted by applying random swaps. For each swap count, we build a BST and an RBT from the resulting list, compute both heights, and record the height difference. We repeat 20 trials per swap count to average out noise. Finally, we graph the number of swaps against the average height difference.

- **List length/Number of nodes:** 1000
- **Swap counts tested:** 0, 1, 2, 5, 10, 25, 50, 100, 200, 500
- **Trials per swap count:** 20

2.3 Results

This experiment gives us 2 major conclusions: BSTs perform horribly on sorted lists, and this difference in performance is almost nonexistent by 500 swaps. The logic is consistent, we see that when we perform little to no swaps, our difference in height is around 800 - 1000, that is, we hit the worst case scenario for the BST where we just create a continuous chain akin to a linked list, since every next element is larger than all previous elements, we get a height of n . Of course, the performance of an RBT is consistently between $\log n$ and $2 \log n$, since we went through all the work of maintaining this balance.

However, as we perform swaps our input list becomes more and more randomized. When we have 500 swaps, we see the difference in height between the BST and RBT is almost non-existent! At this point we consider what we know about RBTs, namely that there is a lot of overhead in managing the structure. Then, if the heights are about the same, the RBT no longer has any advantage over a simple BST. In other words, we have discovered the exact use cases for both types of trees: RBTs provide consistent and reliable results no matter the input, which is especially useful for nearly sorted lists. On the other hand, BSTs in the best case, i.e, mostly random lists, will have roughly the same height as an RBT, and so if we know the list will not be close to sorted, we would prefer to use a BST as it will have less overhead to manage, such as checking structure on every insertion.

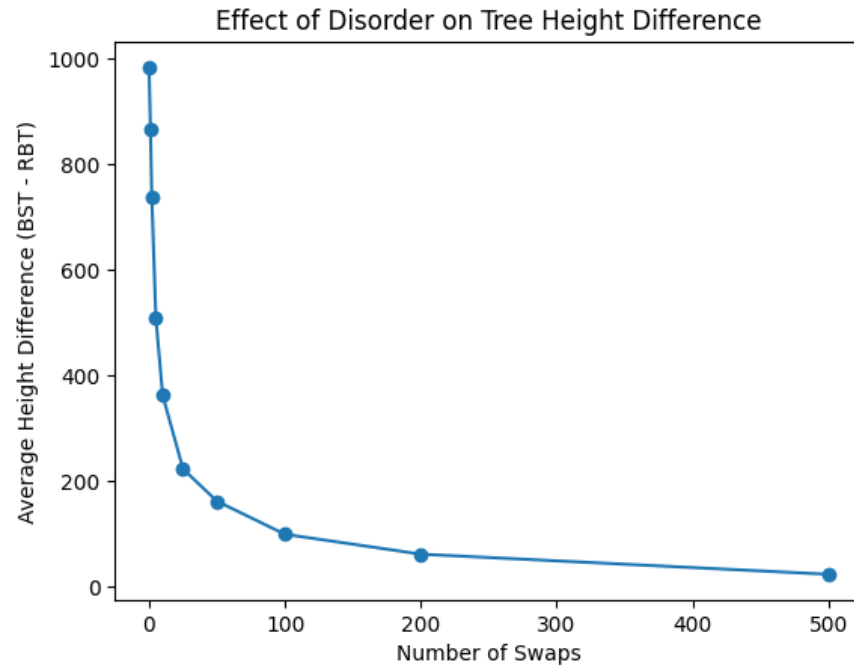


Figure 1: Effect of Disorder on Tree Height Difference for 1000 nodes

3 XC3-Tree Height $h(i)$ for Degrees 0 to 25

3.1 Objective

Our goal is to create XC3-Trees with degrees 0 to 25, and based on the observed heights, write an equation $h(i)$ that returns the height of a degree i XC3-Tree.

3.2 Methodology

We generated XC3-Trees for each degree i from 0 to 25. For each tree, we computed its height using our height function. This function works similar to a normal BST height function: We define it recursively, where the height is 1 + the largest height among all children. Logically, for this tree if the i th child has degree $i - 2$, then the larger the i the larger the degree and thus the larger its height. So, to find the largest height among all children of a node we need to check the the largest child, in other words, the last child. A node's degree is how many children it has, so for example a node with degree k has k children, and the largest child (last one) will have the degree $k - 2$. We used this idea to formalize our function:

```
def xc3_height(deg):
    if deg <= 0:
        return 1
    if deg == 1:
        return 2
    return 1 + xc3_height(deg - 2)
```

We have the 2 base cases where we cannot subtract 2 (negatives). Otherwise, the height is simply 1 + the height of the last child, which will have a degree of $k - 2$ (assuming k is the current degree)

3.3 Results

Now lets look at height. Height we have 1, 2, 2, 3, 3, 4, 4, 5, 5, ... What we can see is height increases every 2 degrees. So:

$$h(0) = 1, \quad h(1) = 2, \quad \text{and} \quad h(i) = 1 + h(i - 2) \text{ for } i \geq 2.$$

This makes perfect sense, this is exactly what our code does and so the mathematical definition just confirms what we suspected. We can even go a step further. We see that barring degree 0, the height increases every 2 degrees, each time increasing by 1. This hints to a ceiling or floor function involved, and after graphing it out and analyzing further, we see that the exact height for a tree with degree i can also be represented as $h(i) = \lceil \frac{i}{2} \rceil + 1$.

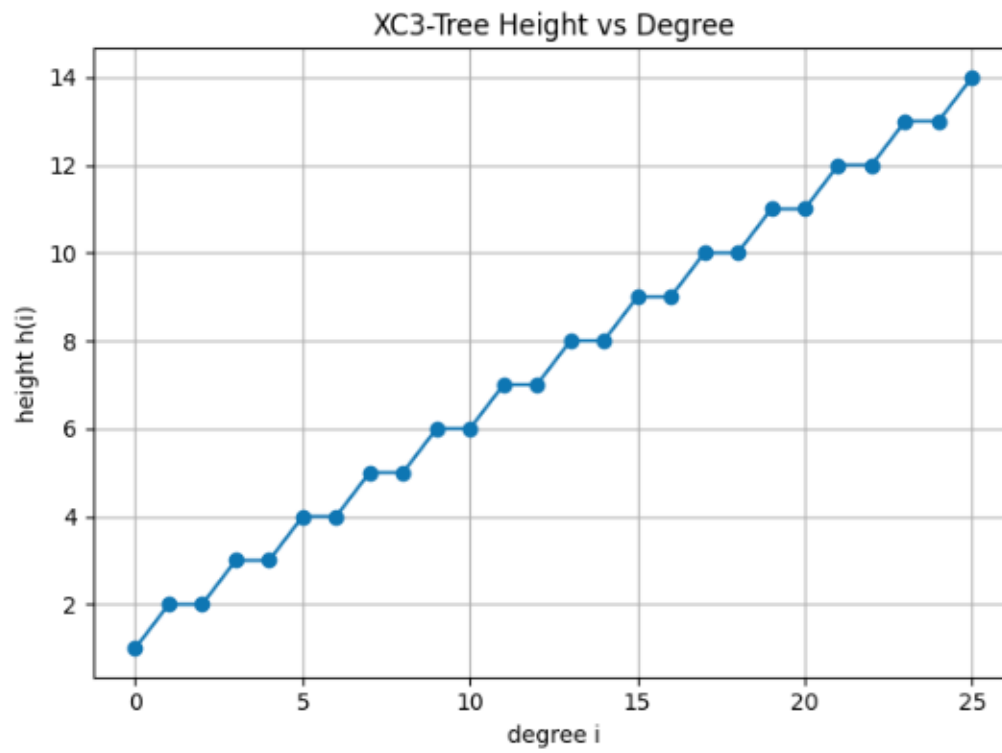


Figure 2: XC3-Tree degree i vs height $h(i)$

4 XC3-Tree Nodes $\text{nodes}(i)$ and Height $O(\log n)$

4.1 Objective

Our goal is to create XC3-Trees with degrees 0 to 25, derive an equation $\text{nodes}(i)$ for the number of nodes in a degree i XC3-Tree, and use this pattern to argue the height is $O(\log n)$.

4.2 Methodology

We generated XC3-Trees for each degree i from 0 to 25 and counted the total number of nodes in each tree. To do this, we used a similar method as for calculating height. We take the same idea as for regular BSTs, where we want to recursively sum all nodes of all children plus the current node.

```
def xc3_nodes(deg):
    if deg <= 0:
        return 1
    if deg == 1:
        return 2
    total = 1
    for k in range(1, deg + 1):
        total += xc3_nodes(k - 2)
    return total
```

4.3 Results

Looking at the output the values follow a Fibonacci sequence $(1, 2, 3, 5, 8, 13, 21, \dots)$. So we can write:

$$\text{nodes}(0) = 1, \quad \text{nodes}(1) = 2, \quad \text{and} \quad \text{nodes}(i) = \text{nodes}(i-1) + \text{nodes}(i-2) \text{ for all } i \geq 2.$$

Why though? When we increase the degree from $i-1$ to i , we add exactly one new child subtree at the end, and that subtree has degree $(i-2)$. So we end up with:

$$\text{nodes}(i) - \text{nodes}(i-1) = \text{nodes}(i-2).$$

Doing a little algebra, we get $\text{nodes}(i) = \text{nodes}(i-1) + \text{nodes}(i-2)$.

4.4 Argument: Why Height is $O(\log n)$

So then comparing this against $O(\log n)$, we know Fibonacci grows exponentially. In particular Fibonacci numbers grow proportionally to the golden ratio (ϕ^i) . That means $\text{nodes}(i)$ grows like ϕ^i , up to a constant factor, let's call it C . If we flip that around we get that a tree with n nodes has $n \geq C\phi^i$, so then using log laws:

$$i \leq \log_{\phi} \left(\frac{n}{C} \right).$$

This is the same as $i \leq \log_\phi(n)$ up to a constant shift. Due to how bases are changed in logarithms, showing that i is $O(\log_\phi n)$ means the same as this being $O(\log n)$ for any other constant base. Therefore we can conclude that since height grows with the degree (it only increases by 1 every time the degree increases by 2), the height is also $O(\log n)$.

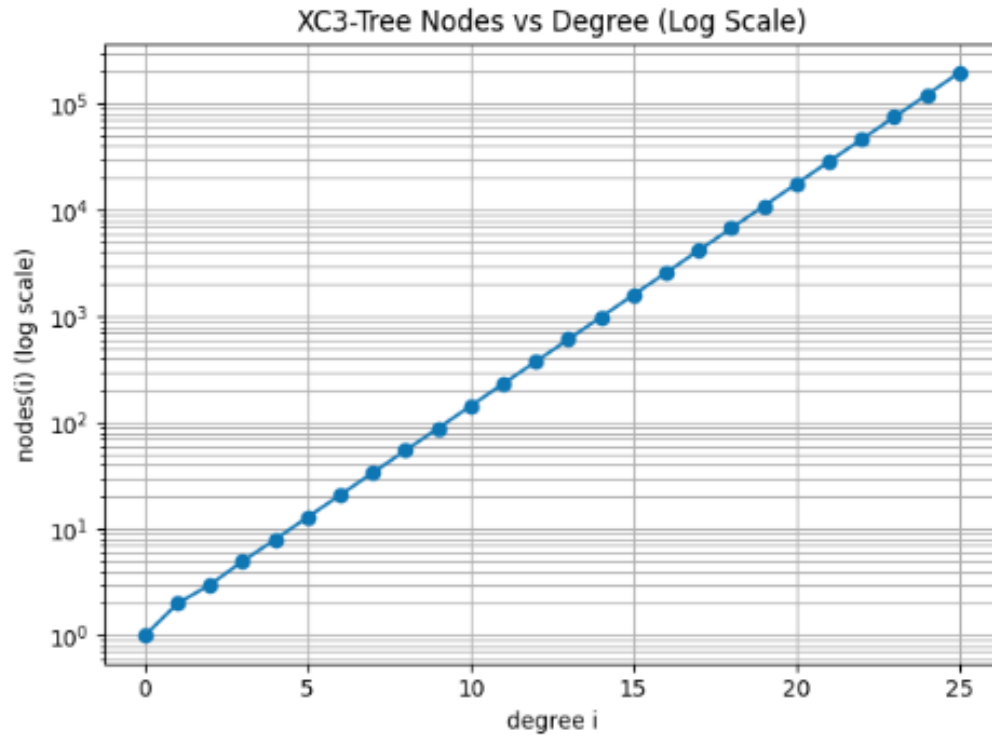


Figure 3: XC3-Tree degree i vs nodes(i) (log scale)

A Code Navigation Guide

To verify the results presented in this report, please refer to the attached source files / notebooks. All code was run on a Python 3.x kernel. **!!BEFORE RUNNING!!** You must have matplotlib and numpy installed on the kernel before running any of the experiments. This can be done by running ‘pip install matplotlib’ and ‘pip install numpy’ in the environment. Here is a guide on what is attached:

- **auxiliary/lab3.py**: Contains the completed implementation of RBTs, with correct rotation methods
- **auxiliary/rbt.py**: Contains the fix() method used for RBT inserts
- **auxiliary/bst.py**: Contains the basic BST implementation used as a comparison baseline for height experiments.
- **experiments/**: Contains the main code for Experiment 1 (random inputs) and Experiment 2, as well as part 2 of the lab (Both XC3 tree experiments)
- **img/**: Contains all relevant graphs from each experiment